

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_excel("bayesian_dataset.xlsx")

print(df.head())
print("\nDataset Shape:", df.shape)

```

	x1	x2	label
0	2.340573	1.558854	0
1	1.825654	0.725435	0
2	2.335983	1.881022	0
3	0.854325	1.847456	0
4	2.600883	4.366434	0

Dataset Shape: (400, 3)

```

X = df[['x1', 'x2']].values
y = df['label'].values

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

```

```

def gaussian_pdf(x, mean, cov):
    size = len(x)
    det = np.linalg.det(cov)
    norm_const = 1 / (np.power((2*np.pi), size/2) * np.sqrt(det))
    x_mu = x - mean
    result = np.exp(-0.5 * x_mu.T @ np.linalg.inv(cov) @ x_mu)
    return norm_const * result

```

```

def bayes_predict(X):
    predictions = []

```

Priors

```

prior0 = np.mean(y_train == 0)
prior1 = np.mean(y_train == 1)

```

Class statistics

```

mean0 = np.mean(X_train[y_train == 0], axis=0)
cov0 = np.cov(X_train[y_train == 0].T)

```

```

mean1 = np.mean(X_train[y_train == 1], axis=0)
cov1 = np.cov(X_train[y_train == 1].T)

```

```

for x in X:
    p0 = gaussian_pdf(x, mean0, cov0) * prior0

```

```

        p1 = gaussian_pdf(x, mean1, cov1) * prior1
        predictions.append(0 if p0 > p1 else 1)

    return np.array(predictions)

y_pred = bayes_predict(X_test)

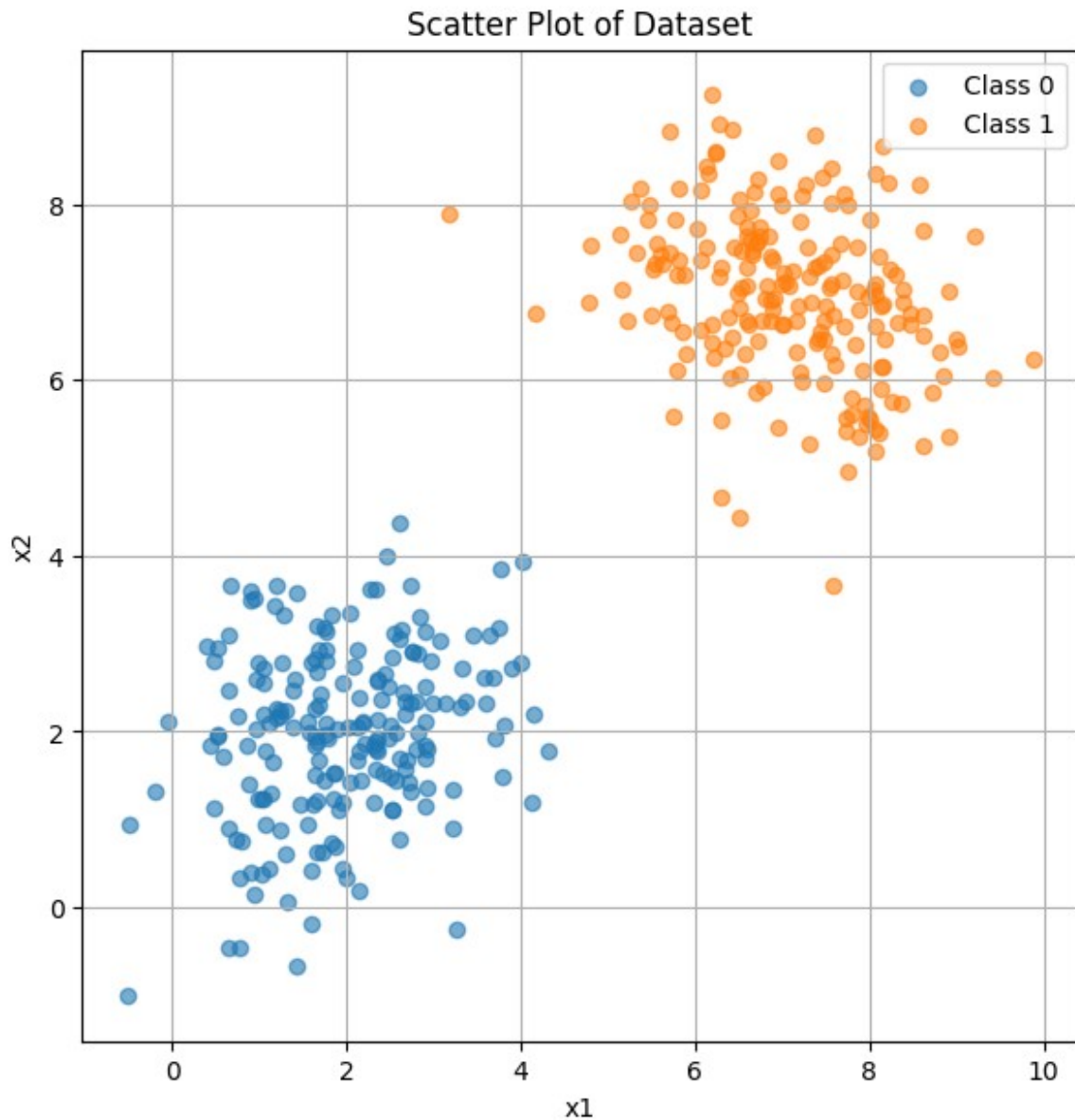
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

Accuracy: 1.0

Confusion Matrix:
[[62  0]
 [ 0 58]]

plt.figure(figsize=(7,7))
plt.scatter(df[df.label==0].x1, df[df.label==0].x2, label="Class 0",
            alpha=0.6)
plt.scatter(df[df.label==1].x1, df[df.label==1].x2, label="Class 1",
            alpha=0.6)
plt.legend()
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Scatter Plot of Dataset")
plt.grid(True)
plt.show()

```



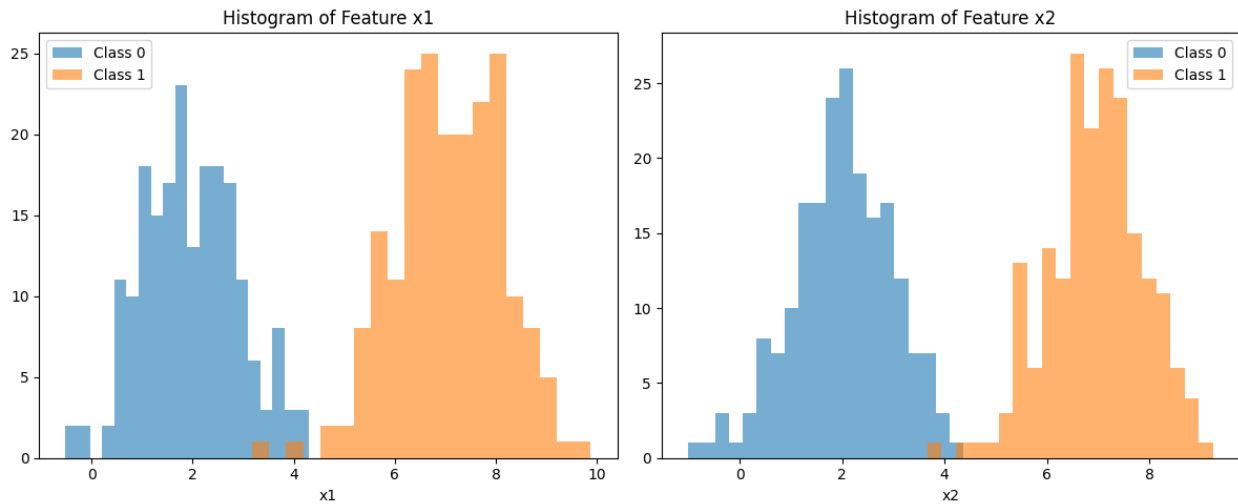
```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.hist(df[df.label==0].x1, alpha=0.6, bins=20, label="Class 0")
plt.hist(df[df.label==1].x1, alpha=0.6, bins=20, label="Class 1")
plt.title("Histogram of Feature x1")
plt.xlabel("x1")
plt.legend()

plt.subplot(1,2,2)
plt.hist(df[df.label==0].x2, alpha=0.6, bins=20, label="Class 0")
plt.hist(df[df.label==1].x2, alpha=0.6, bins=20, label="Class 1")
plt.title("Histogram of Feature x2")
```

```
plt.xlabel("x2")
plt.legend()

plt.tight_layout()
plt.show()
```



```
# Create mesh
x_min, x_max = X[:,0].min() - 1, X[:,0].max() + 1
y_min, y_max = X[:,1].min() - 1, X[:,1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                     np.linspace(y_min, y_max, 200))

grid = np.c_[xx.ravel(), yy.ravel()]
boundary = bayes_predict(grid)

plt.figure(figsize=(7,7))
plt.contourf(xx, yy, boundary.reshape(xx.shape), alpha=0.3)
plt.scatter(X[:,0], X[:,1], c=y, cmap="bwr", alpha=0.7)
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Bayesian Classifier Decision Boundary")
plt.show()
```

Bayesian Classifier Decision Boundary

